

Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω , Θ for sample programs

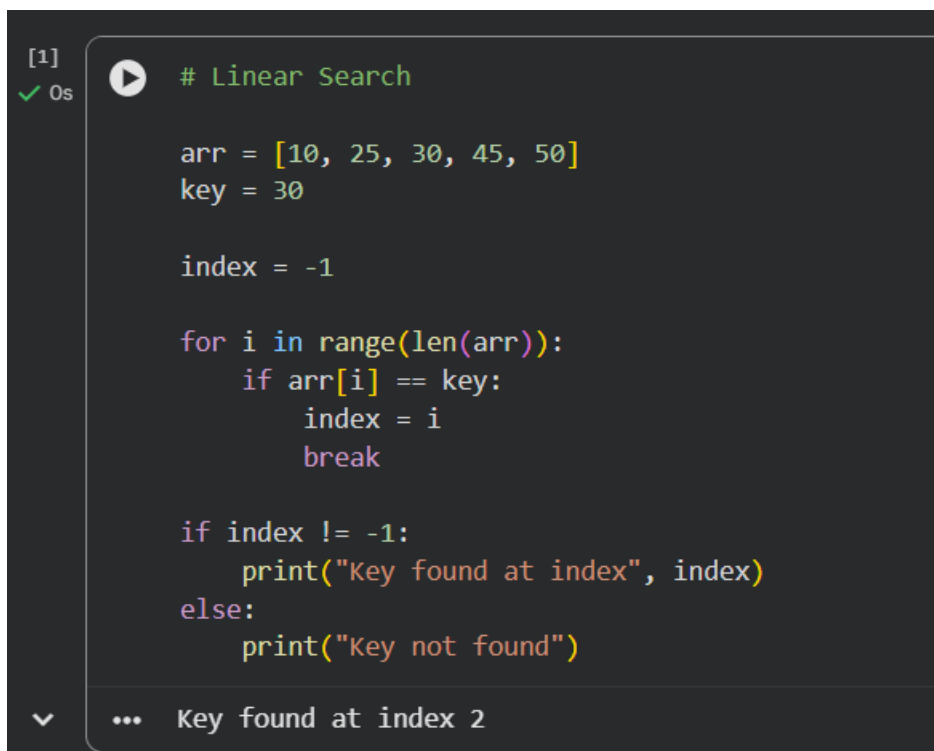
Faculty :Dr. P. SRIRAMYA

Name: PUGAZHARASAN K

Reg. No.: 192511341

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.



```
[1]
✓ 0s # Linear Search

arr = [10, 25, 30, 45, 50]
key = 30

index = -1

for i in range(len(arr)):
    if arr[i] == key:
        index = i
        break

if index != -1:
    print("Key found at index", index)
else:
    print("Key not found")

... Key found at index 2
```

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** $\Omega: O(1)$, $O: O(\log_{10} n)$, $\Theta: \Theta(\log_{10} n)$
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

```
[3]
✓ 0s # Recursive Binary Search

def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] > key:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)
    return -1

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr)-1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")

... Key found at index 3
```

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** $n = 5$
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

```
[4]  # Factorial - Iterative and Recursive
✓ 0s

def recursive_fact(n):
    if n == 0 or n == 1:
        return 1
    return n * recursive_fact(n - 1)

n = 5

# Iterative
fact = 1
for i in range(1, n + 1):
    fact *= i

print("Iterative Factorial:", fact)
print("Recursive Factorial:", recursive_fact(n))

... Iterative Factorial: 120
Recursive Factorial: 120
```

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2^n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

```
[1]
✓ 0s
# Fibonacci Series

def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)

n = 6

print("Iterative Fibonacci:")
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b

print("\n")

print("Recursive Fibonacci:")
for i in range(n):
    print(fib_recursive(i), end=" ")

... Iterative Fibonacci:
0 1 1 2 3 5

Recursive Fibonacci:
0 1 1 2 3 5
```

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- **Sample Output:** $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

```
[2]
✓ Os # Matrix Multiplication

A = [[1, 2],
      [3, 4]]

B = [[5, 6],
      [7, 8]]

result = [[0, 0],
           [0, 0]]

for i in range(2):
    for j in range(2):
        for k in range(2):
            result[i][j] += A[i][k] * B[k][j]

print("Result Matrix:")
for row in result:
    print(row)

... Result Matrix:
[19, 22]
[43, 50]
```

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

```
[3] # Merge Sort
✓ Os

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

arr = [38, 27, 43, 3, 9, 82, 10]

merge_sort(arr)

print("Sorted Array:", arr)
```

... Sorted Array: [3, 9, 10, 27, 38, 43, 82]

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log n)$, O : $O(n^2)$, Θ : $\Theta(n \log n)$

- **Explanation:** Shows how pivot choice affects performance.

```
[5]
✓ 0s # Quick Sort

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return left + middle + right

arr = [10, 7, 8, 9, 1, 5]

print("Sorted Array:", quick_sort(arr))

... Sorted Array: [7, 8, 1, 5, 9, 10]
```

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$

- **Explanation:** Illustrates exponential growth in recursive calls.

```
[6] ✓ Os # Tower of Hanoi

def tower(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower(n-1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    tower(n-1, auxiliary, source, destination)

n = 3

tower(n, 'A', 'B', 'C')
```

▼ ... Move disk 1 from A to C
 Move disk 2 from A to B
 Move disk 1 from C to B
 Move disk 3 from A to C
 Move disk 1 from B to A
 Move disk 2 from B to C
 Move disk 1 from A to C

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log_{10} n)$

- **Explanation:** Shows efficient recursion with logarithmic complexity.

```
[7]
✓ 0s # Euclidean Algorithm

def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a = 48
b = 18

print("GCD =", gcd(a, b))

... GCD = 6
```

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω : $O(n)$, O : $O(n^2)$, Θ : $\Theta(n^2)$

- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

```
[8] ✓ 0s # Insertion Sort

arr = [12, 11, 13, 5, 6]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and key < arr[j]:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted Array:", arr)
```

... Sorted Array: [5, 6, 11, 12, 13]

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$

- **Explanation:** Consistent performance using heap structure.

```
[10] # Heap Sort
✓ 0s

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

arr = [4, 10, 3, 5, 1]

heap_sort(arr)

print("Sorted Array:", arr)
```

... Sorted Array: [1, 3, 4, 5, 10]

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3

- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

```
[11]
✓ 0s # Counting Inversions (Brute Force)

arr = [2, 4, 1, 3, 5]
count = 0

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            count += 1

print("Number of Inversions:", count)

... Number of Inversions: 3
```

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7

- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

```
[12]
✓ 0s # Kadane's Algorithm

arr = [-2, -3, 4, -1, -2, 1, 5, -3]

max_sum = arr[0]
current_sum = arr[0]

for i in range(1, len(arr)):
    current_sum = max(arr[i], current_sum + arr[i])
    max_sum = max(max_sum, current_sum)

print("Maximum Subarray Sum:", max_sum)
```

... Maximum Subarray Sum: 7

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$

- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log_{10} n)$
- **Explanation:** Shows how recursion reduces complexity.

```
[13] 0s # Fast Power

def power(x, n):
    if n == 0:
        return 1

    half = power(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

x = 2
n = 10

print("Result:", power(x, n))

... Result: 1024
```

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.

- **Sample Input:** $[(1, 2), (4, 5), (7, 8), (3, 1)]$
- **Sample Output:** Closest pair $(1, 2) - (3, 1)$, Distance = $\sqrt{5}$
- **Complexity:** Brute force $O(n^2)$, Optimized $O(n \log n)$
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

```
[14]
✓ Os # Closest Pair of Points

import math

points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_dist = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        dist = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

        if dist < min_dist:
            min_dist = dist
            pair = (points[i], points[j])

print("Closest Pair:", pair)
print("Distance:", round(min_dist, 2))
```

... Closest Pair: ((1, 2), (3, 1))
Distance: 2.24